

Config

Lunixose

August 5, 2026

Contents

1	Doom Modules	5
1.1	input	5
1.2	completion	5
1.3	ui	5
1.4	editor	6
1.5	emacs	6
1.6	term	7
1.7	checkers	7
1.8	tools	7
1.9	os	8
1.10	lang	8
1.11	email	10
1.12	app	10
1.13	config	10
2	Personal Information	11
3	Basic	11
3.1	Better Defaults	11
3.2	Editing Defaults	11
3.3	Buffer Behavior	12
4	Appearance	12
4.1	Display	12
4.2	Splash Screen	12
4.3	Theme and Fonts	13

5	Editor	13
5.1	Evil-mode	13
5.2	Mistty	13
5.2.1	Config	14
5.2.2	Keybind	14
5.3	Tabout	14
5.4	Snippets	15
6	Languages	15
6.1	Polymode	15
6.2	LSP	15
6.3	Web	16
6.4	Python	16
6.5	Markdown	16
6.5.1	Titles	16
6.5.2	Autohide	16
6.6	Orgmode	18
6.6.1	Appearance	18
6.6.2	Org Directory	18
6.6.3	Fonts and Headings	19
6.6.4	Emphasis Markers	19
6.6.5	Behaviour	20
6.6.6	Poly-org	21
6.6.7	Agenda	23
7	Note-taking	25
7.1	Install package	25
7.2	Configuration	25
7.2.1	Basic	25
7.2.2	Sequence	26
7.2.3	Journal	26
7.2.4	Browse	26
7.2.5	Keybinds	27
8	HTML Export	27
8.1	Extra header contents	28
8.2	Custom CSS/JS	33
8.3	Collapsible src and example blocks	34
8.4	Handle table overflow	39
8.5	TOC as a collapsable tree	40

8.6	Make verbatim different to code	40
8.7	Change checkbox type	41
8.8	Extra special strings	41
8.9	Header anchors	42
8.10	L ^A T _E X Rendering	42
	8.10.1 Pre-rendered	42
	8.10.2 MathJax	43

This is a literate Doom Emacs configuration

1 Doom Modules

1.1 input

Input sources — none active. Bidi, Chinese, Japanese, and layout all left disabled for now.

```
;;; $DOOMDIR/init.el -*- lexical-binding: t; -*-
;; (doom! :input)
;;bidi                ; (tfel ot) thgir etirw uoy gnipleh
;;chinese
;;japanese
;;layout              ; auie,ctsrnm is the superior home row)
```

1.2 completion

Vertico for minibuffer, Corfu for in-buffer. Both with Orderless for flexible matching.

```
(doom! :completion
      ;; company
      (corfu +orderless)
      ;;helm
      ;;ido
      ;;ivy
      (vertico +icons))
```

1.3 ui

Visual chrome: modeline, popup, ligatures, dashboard, smooth-scroll, unicode, etc.

```
(doom! :ui
  deft                ; notational velocity for Emacs
  doom                ; what makes DOOM look the way it does
  dashboard           ; a nifty splash screen for Emacs
  ;;(emoji +unicode) ;
  hl-todo             ; highlight TODO/FIXME/NOTE/DEPRECATED/HACK/REVIEW
  indent-guides       ; highlighted indent columns
  (ligatures +extra) ; ligatures and symbols to make your code pretty again
  modeline            ; snazzy, Atom-inspired modeline, plus API
  ophints             ; highlight the region an operation acts on
  (popup +defaults) ; tame sudden yet inevitable temporary windows
  smooth-scroll       ; So smooth you won't believe it's not butter
  ;;tabs              ; a tab bar for Emacs
  unicode            ; extended unicode support for various languages
  (vc-gutter +pretty); vcs diff in the fringe
  window-select       ; visually switch windows
  workspaces)         ; tab emulation, persistence & separate workspaces
```

1.4 editor

Evil everywhere, plus formatting, snippets, whitespace, parinfer.

```
(doom! :editor
  (evil +everywhere); come to the dark side, we have cookies
  file-templates   ; auto-snippets for empty files
  fold             ; (nigh) universal code folding
  (format +onsave) ; automated prettiness
  ;;god            ; run Emacs commands without modifier keys
  ;;lispy          ; vim for lisp, for people who don't like vim
  ;; multiple-cursors ; editing in many places at once
  ;;objed
  parinfer          ; turn lisp into python, sort of
  rotate-text       ; cycle region between text candidates
  snippets          ; my elves. They type so I don't have to
  (whitespace +guess +trim)) ; a butler
  ;;word-wrap       ; soft wrapping with language-aware indent
```

1.5 emacs

Built-in Emacs conveniences wrapped by direx, eww, ibuffer, tramp, undo, vc.

```
(doom! :emacs
      dired          ; making dired pretty [functional]
      electric       ; smarter, keyword-based electric-indent
      eww            ; the internet is gross
      (ibuffer +icons) ; interactive buffer management
      tramp          ; remote files at your arthritic fingertips
      undo           ; persistent, smarter undo for your inevitable mistakes
      vc)            ; version-control and Emacs, sitting in a tree
```

1.6 term

Eshell only inside the doom `:term` block. Mistty handles the in-Emacs terminal (see the Mistty section below). Plain `term` left disabled.

```
(doom! :term
      eshell)          ; the elisp shell that works everywhere
      ;;shell          ; simple shell REPL for Emacs
      ;;term           ; basic terminal emulator for Emacs
      ;; vterm)        ; the best terminal emulation in Emacs (no longer used)
```

1.7 checkers

Syntax checker only. Spell + grammar left disabled.

```
(doom! :checkers
      syntax)          ; tasing you for every
      ;; (spell +flyspell) ; tasing you for misspelling misspelling
      ;; grammar        ; tasing grammar mistake every you make)
```

1.8 tools

External tool integrations: LSP, magit, docker, pass, tree-sitter, etc. (`eval +overlay`) is present but `SPC e` was reassigned to Denote.

```
(doom! :tools
      ;; ansible
      biblio          ; Writes a PhD for you (citation needed)
      ;;collab        ; buffers with friends
      ;; debugger      ; stepping through code, to help you add bugs
      ;; direnv
      (docker +tree-sitter)
```

```

editorconfig      ; let someone else argue about tabs vs spaces
;;ein             ; tame Jupyter notebooks with emacs
(eval +overlay)   ; run code, run (also, repls)
lookup            ; navigate your code and its documentation
;;llm             ; when I said you needed friends, I didn't mean...
lsp               ; M-x vscode
(magit +forge)    ; a git porcelain for Emacs
;; make           ; run make tasks from Emacs
pass              ; password manager for nerds
pdf               ; pdf enhancements
;;terraform       ; infrastructure as code
;; tmux           ; an API for interacting with tmux
tree-sitter       ; syntax and parsing, sitting in a tree...
upload            ; map local to remote projects via ssh/ftp

```

1.9 os

Mac-specific tweaks only on macOS. TTY left disabled.

```

(doom! :os
  (:if (featurep :system 'macos) macos) ; improve compatibility with macOS
  tty) ; improve the terminal Emacs experience)

```

1.10 lang

Languages with LSP and tree-sitter where supported. Many commented out as unused.

```

(doom! :lang
  ;;ada ; In strong typing we (blindly) trust
  (agda +local) ; types of types of types of types...
  ;;beancount ; mind the GAAP
  (cc +lsp) ; C > C++ == 1
  ;;clojure ; java with a lisp
  common-lisp ; if you've seen one lisp, you've seen them all
  ;;coq ; proofs-as proofs
  ;;crystal ; ruby at the speed of c
  ;;csharp ; unity, .NET, and mono shenanigans
  data ; config/data formats
  ;;(dart +flutter) ; paint ui and not much else
  ;;dhall

```

```

;;elixir          ; erlang done right
;;elm             ; care for a cup of TEA?
emacs-lisp        ; drown in parentheses
;;erlang          ; an elegant language for a more civilized age
;;ess             ; emacs speaks statistics
;;factor
;;faust           ; dsp, but you get to keep your soul
;;fortran         ; in FORTRAN, GOD is REAL (unless declared INTEGER)
;;fsharp          ; ML stands for Microsoft's Language
;;fstar           ; (dependent) types and (monadic) effects and Z3
;;gdscrip         ; the language you waited for
(go +lsp)         ; the hipster dialect
;;(graphql +lsp)  ; Give queries a REST
;;(haskell +lsp) ; a language that's lazier than I am
;;hy              ; readability of scheme w/ speed of python
;;idris           ; a language you can depend on
json              ; At least it ain't XML
;;janet           ; Fun fact: Janet is me!
(java +lsp)       ; the poster child for carpal tunnel syndrome
javascript        ; all(hope(abandon(ye(who(enter(here)))))))
;;julia           ; a better, faster MATLAB
;;kotlin          ; a better, slicker Java(Script)
(latex +lsp +fold +cdlatex) ; writing papers in Emacs has never been
lean              ; for folks with too much to prove
;;ledger          ; be audit you can be
(lua +lsp)        ; one-based indices? one-based indices
(markdown +lsp +tree-sitter +grip) ; writing docs for people to ignore
;;nim             ; python + lisp at the speed of c
(nix +lsp +tree-sitter) ; I hereby declare "nix geht mehr!"
;;ocaml           ; an objective camel
;;odin            ; C, minus its footguns
(org +pandoc)     ; organize your plain life in plain text
php              ; perl's insecure younger brother
;;plantuml        ; diagrams for confusing people more
;;graphviz        ; diagrams for confusing yourself even more
;;purescript      ; javascript, but functional
(python +lsp +tree-sitter) ; beautiful is better than ugly
(qt +lsp +tree-sitter)    ; the 'cutest' gui framework ever
;;racket          ; a DSL for DSLs
;;raku            ; the artist formerly known as perl6

```

```

;;rest                ; Emacs as a REST client
;;rst                 ; ReST in peace
(ruby +rails)         ; 1.step {|i| p "Ruby is #{i.even? ? 'love' : 'life'}}"}
(rust +lsp)           ; Fe203.unwrap().unwrap().unwrap().unwrap()
;;scad                ; trust the preview, regret the render
;;scala               ; java, but good
;; (scheme +guile)    ; a fully conniving family of lisps
(sh +lsp)              ; she sells {ba,z,fi}sh shells on the C xor
;;sml
;;solidity            ; do you need a blockchain? No.
;;swift               ; who asked for emoji variables?
;;terra               ; Earth and Moon in alignment for performance.
(web +lsp +tree-sitter) ; the tubes
yaml)                 ; JSON, but readable
;;zig                 ; C, but simpler)

```

1.11 email

mu4e wired to Gmail. Notmuch and Wanderlust left disabled.

```

(doom! :email
  ;;notmuch
  ;;(wanderlust +gmail))
  (wanderlust +gmail))

```

1.12 app

Standalone applications: calendar, IRC, RSS reader.

```

(doom! :app
  calendar
  ;;emms
  ;;everywhere      ; *leave* Emacs!? You must be joking
  irc                ; how neckbeards socialize
  (rss +org))        ; emacs as an RSS reader

```

1.13 config

Literate config (config.org) with default Doom bindings and smartparens.

```

(doom! :config

```



```
literate
(default +bindings +smartparens))
```

2 Personal Information

Some packages (Magit commits, mu4e, snippets, GPG) read these variables to know who you are. Fill in the email below — it's left as a TODO so it doesn't ship a placeholder address.

```
(setq user-full-name "Lunixose"
      user-mail-address "lunixose@gmail.com")
```

3 Basic

Core settings that shape the everyday editing experience: theme, fonts, how the frame opens, and a handful of quality-of-life tweaks.

3.1 Better Defaults

```
(setq-default delete-by-moving-to-trash t
              window-combination-resize t
              x-stretch-cursor t)
(setq undo-limit (* 80 1024 1024)      ; 80 MB of undo history
      evil-want-fine-undo t
      truncate-string-ellipsis "... "
      password-cache-expiry nil
      display-time-default-load-average nil
      which-key-idle-delay 0.4)
(display-time-mode 1)
(global-subword-mode 1)
```

3.2 Editing Defaults

Indent with two spaces and disable Emacs' auto-save (`#file#`) files — version control covers recovery here.

```
(setq-default tab-width 2)
(setq-default evil-shift-width 2)
(setq-default indent-tabs-mode nil)
(setq auto-save-default nil)
```

3.3 Buffer Behavior

Auto-revert keeps buffers in sync with changes (`global-auto-revert-mode` also covers non-file buffers like Dired). The `defadvice` prompts for which buffer to show whenever a window is split, instead of duplicating the current one.

```
(global-auto-revert-mode 1)
(setq global-auto-revert-non-file-buffers t)
(defadvice! prompt-for-buffer (&rest _)
  :after 'window-split (switch-to-buffer))
```

4 Appearance

4.1 Display

Open maximized and use relative line numbers, which pair well with Evil's count-based motions.

```
(setq display-line-numbers-type 'relative)
```

4.2 Splash Screen

```
;; Place 'my-banner.png' inside your ~/.config/doom/ directory
;; (setq fancy-splash-image (expand-file-name "my-banner.png" doomdir))
(defun my-custom-ascii-banner-fn ()
  (propertize
    "
      --  --      ----
      |  \\\\/  |_  _|  _ \\\  _ _ _ _ _ _ _ _ _ _
      |  \\\\/|  |  |  |  |  | /  _ \\\  /  _ \\\|  '  '  _ \\\
      |  |  |  |  |  |  |  | ( )  | ( )  |  |  |  |  |
      |  |  |  |  \\\_,  |____/  \\\____/  \\\____/|  |  |  |  |
                        |____/
    "
    'face '+dashboard-banner))
(setq +dashboard-ascii-banner-fn #'my-custom-ascii-banner-fn)

(defun my-dashboard-widget-welcome ()
  "Inserts a welcome message into the dashboard."
  (insert "\n"))
```

```
(+dashboard-insert
  (propertize "Welcome home, LUnixose!"
    ;; Try 'font-lock-string-face or 'font-lock-comment-face
    'face 'font-lock-keyword-face)))

(setq +dashboard-functions
  '(+dashboard-widget-banner
    my-dashboard-widget-welcome
    +dashboard-widget-footer
    +dashboard-widget-loaded))
```

4.3 Theme and Fonts

Pick the Monokai Octagon theme and set JetBrainsMono Nerd Font as the default. The Nerd Font variant ships the glyphs used by the modeline and dashboard icons.

```
(setq doom-theme 'doom-monokai-octagon)
(setq doom-font (font-spec :family "JetBrainsMono Nerd Font" :size 18 :weight 'medium))
```

5 Editor

General editing behaviour that isn't tied to a single language.

5.1 Evil-mode

Use emacs mode as editing mode

```
(setq! evil-disable-insert-state-bindings t)
(defalias 'evil-insert-state 'evil-emacs-state)
(define-key evil-emacs-state-map (kbd "<escape>") 'evil-normal-state)
(define-key evil-emacs-state-map (kbd "C-g") 'evil-normal-state)
```

5.2 Mistty

Mistty is an in-Emacs terminal that uses Emacs' built-in `comint` (“*emulator*” mode) instead of `libvterm`. No C dependency; uses `bash` as the shell. Replaces `vterm` (which has been removed from the `:term` block above).

```
(package! mistty
  :recipe (:host github :repo "szermatt/mistty") :pin "4d32106dc2a60183c0df8716502ed404")
```

5.2.1 Config

```
(after! mistty
  (setq mistty-process-type 'emulator
        mistty-shell-command "nu"
        mistty-shell "/usr/bin/env nu"
        mistty-shell-arg '("-i")
        mistty-width 80
        mistty-height 24))
```

5.2.2 Keybind

```
(map! :leader
  (:prefix ("v" . "Term")
    :desc "Open in project"      "t" #'mistty-in-project
    :desc "Open in project"      "o" #'mistty
    :desc "sudo"                  "s" #'mistty-sudo
    :desc "SSH"                   "h" #'mistty-ssh
    :desc "Send key"              "q" #'mistty-send-key
    :desc "Mistty clear"          "c" #'mistty-clear
    :desc "Next input"            "n" #'mistty-next-input
    :desc "Previous input"        "p" #'mistty-previous-input))
```

5.3 Tabout

Repurpose TAB in insert mode to navigate bracket pairs dynamically (tab-in, between, out) with graceful fallback to indent or snippet expansion.

- package:

```
(package! lnav
  :recipe (:host github :repo "quocjq/lnav")
  :pin "e9e6b626d4e4c0f57518724c20662528410e911f")
```

- config:

```
(map! :e "M-]" #'lnav-next-chunk
  :e "M-[" #'lnav-previous-chunk
  :n "M-]" #'lnav-next-chunk
  :n "M-[" #'lnav-previous-chunk)
```

5.4 Snippets

Doom ships YASnippet. This points it at a personal snippets directory and lets one snippet expand into another (nested expansion), which is handy for building larger templates out of small pieces.

```
(after! yasnippet
  (setq yas-triggers-in-field t)          ; allow nested snippet expansion
  (add-to-list 'yas-snippet-dirs (expand-file-name "snippets" doom-user-dir)))
```

See snippets-guide.org for the snippet file format, placeholders, and how to add new ones.

6 Languages

Per-language tweaks. Doom's `:lang` modules already do the heavy lifting (LSP, tree-sitter, formatting) — these just adjust a few defaults, in the same spirit as `tecosaur`'s language section but pared down to the languages you have enabled.

6.1 Polymode

- Install Polymode

```
(package! polymode
  :recipe (:host github :repo "polymode/polymode") :pin "8cb72fa5dcc0d98746c680")
```

6.2 LSP

Trim the LSP client for responsiveness: raise the debounce a touch so completion isn't recomputed on every keystroke, and stop it from scanning huge trees of files for changes. Disabling the doc-glance popup keeps hovers from stealing focus.

```
(after! lsp-mode
  (setq lsp-idle-delay 0.5
        lsp-enable-file-watchers nil
        lsp-enable-symbol-highlighting nil))
(after! lsp-ui
  (setq lsp-ui-doc-enable nil
        lsp-ui-sideline-enable nil))
```

6.3 Web

Two-space indentation for HTML/CSS/JS, which matches your global `tab-width` and the common convention for web files.

```
(after! web-mode
  (setq web-mode-markup-indent-offset 2
        web-mode-css-indent-offset 2
        web-mode-code-indent-offset 2))
```

6.4 Python

Silence the native-completion warning that pops up when the running Python lacks readline, and prefer `pytest` as the test runner.

```
(after! python
  (setq python-shell-completion-native-enable nil))
```

6.5 Markdown

6.5.1 Titles

```
(custom-set-faces!
  '(markdown-header-delimiter-face :foreground "#616161" :height 0.9)
  '(markdown-header-face-1 :height 1.8 :foreground "#A3BE8C" :weight extra-bold :inherit markdown-header-face-1)
  '(markdown-header-face-2 :height 1.4 :foreground "#EBCB8B" :weight extra-bold :inherit markdown-header-face-1)
  '(markdown-header-face-3 :height 1.2 :foreground "#D08770" :weight extra-bold :inherit markdown-header-face-1)
  '(markdown-header-face-4 :height 1.15 :foreground "#BF616A" :weight bold :inherit markdown-header-face-1)
  '(markdown-header-face-5 :height 1.1 :foreground "#b48ead" :weight bold :inherit markdown-header-face-1)
  '(markdown-header-face-6 :height 1.05 :foreground "#5e81ac" :weight semi-bold :inherit markdown-header-face-1))
```

6.5.2 Autohide

```
(defvar nb/c "(start . end) of current line in current buffer")
(make-variable-buffer-local 'nb/current-line)
(defun my/markdown-unhide-table (limit)
  (when (re-search-forward "^[\\t]*|.*$" limit t)
    (remove-text-properties (match-beginning 0) (match-end 0)
                          '(invisible t display "" composition ""))
    t))

(defun my/markdown-pad-table (limit)
```

```

(when (re-search-forward "^[ \t]*|.*$" limit t)
  (let ((row-end (match-end 0))
        (cell-start (match-beginning 0)))
    (goto-char cell-start)
    (while (search-forward "|" row-end t)
      (let ((pipe-pos (1- (point)))
            (hidden 0))
        (save-excursion
          (goto-char cell-start)
          (while (< (point) pipe-pos)
            (when (or (get-text-property (point) 'invisible)
                      (equal (get-text-property (point) 'display) ""))
              (setq hidden (1+ hidden)))
            (forward-char 1)))
        (when (> hidden 0)
          (put-text-property pipe-pos (1+ pipe-pos) 'display
                              (concat (make-string hidden ?\s) "|")))
        (setq cell-start (point))))
    (goto-char row-end))
  t))

(defun nb/unhide-current-line (limit)
  "Font-lock function"
  (let ((start (max (point) (car nb/current-line)))
        (end (min limit (cdr nb/current-line))))
    (when (< start end)
      (remove-text-properties start end
                              '(invisible t display "" composition ""))
      (goto-char limit)
      t)))

(defun nb/refontify-on-linemove ()
  "Post-command-hook"
  (let* ((start (line-beginning-position))
         (end (line-beginning-position 2))
         (needs-update (not (equal start (car nb/current-line)))))
    (setq nb/current-line (cons start end))
    (when needs-update
      (font-lock-fontify-block 3))))

```

```
(defun nb/markdown-unhighlight ()
  "Enable markdown concealling"
  (interactive)
  (markdown-toggle-markup-hiding 'toggle)
  (font-lock-add-keywords nil '((nb/unhide-current-line)
                                (my/markdown-pad-table)) t)
  (add-hook 'post-command-hook #'nb/refontify-on-linemove nil t))
(add-hook 'markdown-mode-hook #'nb/markdown-unhighlight)
```

6.6 Orgmode

6.6.1 Appearance

`org-modern` restyles headings, lists, tables, keywords, and timestamps for a cleaner, more typographic look. Beyond just enabling it, these settings pick a nicer bullet for lists, draw a fringe marker beside source blocks, and give TODO/DONE labels pill-shaped faces — the visual polish `tecosaur` is known for, without the custom SVG machinery.

```
(package! org-modern)

(after! org
  (setq org-modern-star 'replace ; swap heading asterisks for bullets
        org-modern-list '((?- . "●") (?+ . "○") (?* . ""))
        org-modern-block-fringe 15 ; keep blocks aligned with text
        org-modern-table t
        org-modern-timestamp t)
  ;; Nicer folding ellipsis and pretty entities (\alpha -> , etc.)
  (setq org-ellipsis " "
        org-pretty-entities t
        org-hide-emphasis-markers t)
  (global-org-modern-mode))
```

6.6.2 Org Directory

Where Org files live by default. Agenda, capture, and refile all read from here.

```
(setq org-directory (expand-file-name "~/Documents/notes/"))
```


6.6.3 Fonts and Headings

```
(add-hook 'org-mode-hook #'org-pretty-mode)

(custom-set-faces!
  '(outline-1 :weight extra-bold :height 1.3)
  '(outline-2 :weight bold :height 1.25)
  '(outline-3 :weight bold :height 1.15)
  '(outline-4 :weight semi-bold :height 1.09)
  '(outline-5 :weight semi-bold :height 1.06)
  '(outline-6 :weight semi-bold :height 1.03)
  '(outline-8 :weight semi-bold)
  '(outline-9 :weight semi-bold))
(custom-set-faces!
  '(org-document-title :height 1.5))
(after! org
  (setq org-fontify-quote-and-verse-blocks t
        org-agenda-deadline-faces
        '((1.001 . error)
          (1.0 . org-warning)
          (0.5 . org-upcoming-deadline)
          (0.0 . org-upcoming-distant-deadline))))
```

6.6.4 Emphasis Markers

With `org-hide-emphasis-markers` on (above), the `/slashes/` and `*stars*` around emphasized text are hidden — nice to read, awkward to edit. `org-appear` restores them automatically whenever the cursor enters the emphasized region, so editing stays easy.

```
(package! org-appear
  :recipe (:host github :repo "awth13/org-appear") :pin "77d23efec5f5c25fc079836")

(use-package! org-appear
  :hook (org-mode . org-appear-mode)
  :config
  (setq org-appear-autoemphasis t
        org-appear-autolinks t
        org-appear-autosubmarkers t))
```

```

(setq org-appear-trigger 'manual)
(add-hook 'org-mode-hook (lambda ()
  (add-hook 'evil-emacs-state-entry-hook
    #'org-appear-manual-start
    nil
    t)
  (add-hook 'evil-emacs-state-exit-hook
    #'org-appear-manual-stop
    nil
    t)))

```

6.6.5 Behaviour

```

(after! org
  (setq org-log-done 'time
    org-startup-folded 'content
    org-startup-indented t
    org-hide-leading-stars t
    org-return-follows-link t ; RET opens the link under point
    org-use-property-inheritance t ; properties apply to child headings
    org-list-allow-alphabetical t ; a. A. a) A) list bullets
    org-catch-invisible-edits 'smart ; don't silently edit folded text
    org-export-with-sub-superscripts '{ } ; require x_{i}, not x_i
    org-image-actual-width '(0.9)) ; inline images at 90% of window

```

By default Doom turns `visual-line-mode` on in text buffers. That wraps in a way that fights Org tables and $\text{\LaTeX}$, so swap it for `auto-fill-mode` (hard wrapping) here — you can still toggle visual wrap per buffer. Also add arrow-key equivalents for Org's `hjkl` structural motions.

```

(remove-hook 'text-mode-hook #'visual-line-mode)
(add-hook 'text-mode-hook #'auto-fill-mode)

(map! :map evil-org-mode-map
  :after evil-org
  :n "g <up>" #'org-backward-heading-same-level
  :n "g <down>" #'org-forward-heading-same-level
  :n "g <left>" #'org-up-element
  :n "g <right>" #'org-down-element)

```

6.6.6 Poly-org

Vendored from upstream `poly-org.el` (Vitalie Spinu, MIT) under our own name so we can wire `parinfer` (and any future `emacs-lisp-mode-hook` consumer) into the `emacs-lisp` `src-block` inner buffer. Polymode uses `delay-mode-hooks`-style deferral when cloning inner buffers, so the plain `emacs-lisp-mode-hook` never fires inside Org `src` blocks; the hook is invoked explicitly from a `switch-buffer-functions` slot.

```
(use-package parinfer-rust-mode
  :hook emacs-lisp-mode
  :init
  (setq parinfer-rust-auto-download t))

;;; my-poly-org.el --- Vendored poly-org, renamed. -*- lexical-binding: t -*-
;;;
(require 'polymode)
(require 'org)
(require 'org-src)

(defun my-poly-org-mode-matcher ()
  (let ((case-fold-search t))
    (when (re-search-forward "#\\++begin_\\(src\\|example\\|export\\) +\\([^\t\n]+\\)"
      (let ((lang (match-string-no-properties 2)))
        (or (cdr (assoc lang org-src-lang-modes))
            lang))))))

(defvar ess-local-process-name)
(defun my-poly-org-convey-src-block-params-to-inner-modes (_ this-buf)
  "Move src block parameters to innermode specific locals.
Used in :switch-buffer-functions slot."
  (cond
    ((derived-mode-p 'ess-mode)
     (with-current-buffer (pm-base-buffer)
       (let* ((params (nth 2 (org-babel-get-src-block-info t)))
              (session (cdr (assq :session params))))
         (when (and session (org-babel-comint-buffer-livep session))
           (let ((proc (buffer-local-value 'ess-local-process-name
                                           (get-buffer session))))
             (with-current-buffer this-buf
               (setq-local ess-local-process-name proc)))))))))
```

```

((derived-mode-p 'emacs-lisp-mode)
 (with-current-buffer (pm-base-buffer)
  (let ((params (nth 2 (org-babel-get-src-block-info t))))
    (with-current-buffer this-buf
      (setq-local lexical-binding
                    (string-equal "t" (cdr (assq :lexical params)))))
      ;; Activate parinfer inside emacs-lisp src blocks.
      (parinfer-rust-mode 1))))))

(define-hostmode my-poly-org-hostmode
  :mode 'org-mode
  :protect-syntax nil
  :protect-font-lock nil)

(define-auto-innermode my-poly-org-innermode
  :fallback-mode 'host
  :head-mode 'host
  :tail-mode 'host
  :head-matcher "^[ \\t]*#\\+begin_\\(src\\|example\\|export\\) .*\\n"
  :tail-matcher "^[ \\t]*#\\+end_\\(src\\|example\\|export\\)"
  :mode-matcher #'my-poly-org-mode-matcher
  :head-adjust-face nil
  :switch-buffer-functions '(my-poly-org-convey-src-block-params-to-inner-modes)
  :body-indent-offset 'org-edit-src-content-indentation
  :indent-offset 'org-edit-src-content-indentation)

(define-innermode my-poly-org-latex-innermode nil
  "Innermode for matching latex fragments in 'org-mode'"
  :mode 'latex-mode
  :body-indent-offset 'LaTeX-indent-level
  :head-matcher "^[ \\t]*\\\\begin{.+}.*$"
  :tail-matcher "^[ \\t]*\\\\end{.+}.*$"
  :head-mode 'host
  :tail-mode 'host)

(define-polymode my-poly-org-mode
  :hostmode 'my-poly-org-hostmode
  :innermodes '(my-poly-org-innermode my-poly-org-latex-innermode)
  (setq-local org-src-fontify-natively t)
  (setq-local polymode-run-these-before-change-functions-in-other-buffers

```

```

      (append '(org-before-change-function
                org-element--cache-before-change
                org-table-remove-rectangle-highlight)
              polymode-run-these-before-change-functions-in-other-buffers))
    (setq-local polymode-run-these-after-change-functions-in-other-buffers
      (append '(org-element--cache-after-change
                org-indent-refresh-maybe)
              polymode-run-these-after-change-functions-in-other-buffers)))

(pm-around-advice 'org-element-at-point #'polymode-with-current-base-buffer)

;;;###autoload
(add-to-list 'auto-mode-alist '("\\.org\\'" . my-poly-org-mode))

```

6.6.7 Agenda

tecosaur’s souped-up agenda, built on `org-super-agenda`. This groups the `alltodo` view by tag, priority, and deadline so a single SPC o A o “Overview” gives you Today’s time-grid plus everything else bucketed sensibly.

```

(package! org-super-agenda)

(use-package! org-super-agenda
  :commands org-super-agenda-mode)

```

The custom command below is tecosaur’s, verbatim. The `:discard` at the end drops chore/routine/daily-tagged items from the overview so it stays focused on real work.

```

(after! org-agenda
  (let ((inhibit-message t))
    (org-super-agenda-mode)))

(setq org-agenda-skip-scheduled-if-done t
      org-agenda-skip-deadline-if-done t
      org-agenda-include-deadlines t
      org-agenda-block-separator nil
      org-agenda-tags-column 100 ;; from testing this seems to be a good value
      org-agenda-compact-blocks t)

(setq org-agenda-custom-commands

```

```

'(("o" "Overview"
  ((agenda "" ((org-agenda-span 'day)
    (org-super-agenda-groups
      '(:name "Today"
        :time-grid t
        :date today
        :todo "TODAY"
        :scheduled today
        :order 1))))))
  (alltodo "" ((org-agenda-overriding-header "")
    (org-super-agenda-groups
      '(:name "Next to do"
        :todo "NEXT"
        :order 1)
      (:name "Important"
        :tag "Important"
        :priority "A"
        :order 6)
      (:name "Due Today"
        :deadline today
        :order 2)
      (:name "Due Soon"
        :deadline future
        :order 8)
      (:name "Overdue"
        :deadline past
        :face error
        :order 7)
      (:name "Assignments"
        :tag "Assignment"
        :order 10)
      (:name "Issues"
        :tag "Issue"
        :order 12)
      (:name "Emacs"
        :tag "Emacs"
        :order 13)
      (:name "Projects"
        :tag "Project"
        :order 14)

```

```

(:name "Research"
 :tag "Research"
 :order 15)
(:name "To read"
 :tag "Read"
 :order 30)
(:name "Waiting"
 :todo "WAITING"
 :order 20)
(:name "University"
 :tag "uni"
 :order 32)
(:name "Trivial"
 :priority<= "E"
 :tag ("Trivial" "Unimportant")
 :todo ("SOMEDAY" )
 :order 90)
(:discard (:tag ("Chore" "Routine" "Daily")))))))))))

```

7 Note-taking

7.1 Install package

```

(package! denote)
(package! denote-sequence)
(package! denote-journal)
(package! consult-denote)

```

7.2 Configuration

7.2.1 Basic

```

(use-package denote
  :ensure t
  :hook
  ((text-mode . denote-fontify-links-mode)
   (dired-mode . denote-dired-mode))

  :config
  (setq denote-directory (expand-file-name "~/Documents/notes/"))
  (setq denote-save-buffers nil)

```

```

(setq denote-known-keywords '("emacs" "nixos" "personal" "learning" "thinking" "philosophy"))
(setq denote-infer-keywords t)
(setq denote-sort-keywords t)
(setq denote-prompts '(title keywords))
(setq denote-excluded-directories-regexp nil)
(setq denote-keywords-to-not-infer-regexp nil)
(setq denote-rename-confirmations '(rewrite-front-matter modify-file-name))
(setq denote-date-prompt-use-org-read-date t)
(denote-rename-buffer-mode 1))

```

7.2.2 Sequence

```

(use-package denote-sequence
  :ensure t
  :after denote
  :config
  (setq denote-sequence-scheme 'alphanumeric))

```

7.2.3 Journal

```

(use-package denote-journal
  :ensure t
  :after denote
  :commands (denote-journal-new-entry
              denote-journal-new-or-existing-entry
              denote-journal-link-or-create-entry)
  :hook (calendar-mode . denote-journal-calendar-mode)
  :config
  (setq denote-journal-directory
        (expand-file-name "journal" denote-directory))
  (setq denote-journal-keyword "journal")
  (setq denote-journal-title-format 'day-date-month-year))

```

7.2.4 Browse

```

(use-package consult-denote
  :ensure t
  :after denote
  :config
  (consult-denote-mode 1)
  (setq consult-denote-grep-command #'consult-ripgrep))

```


7.2.5 Keybinds

```
(after! doom
  (map! :leader
    (:prefix ("e" . "Notes (denote)")
      :desc "new note"          "n" #'denote
      :desc "find (fuzzy)"      "f" #'consult-denote-find
      :desc "grep (ripgrep)"    "g" #'consult-denote-grep
      :desc "link"              "l" #'denote-link
      :desc "add links"         "L" #'denote-add-links
      :desc "backlinks"         "b" #'denote-backlinks
      :desc "rename file"       "r" #'denote-rename-file
      :desc "rename from FM"    "R" #'denote-rename-file-using-front-matter
    )
    (:prefix ("q" . "Query")
      :desc "by contents"      "c" #'denote-query-contents-link
      :desc "by filenames"     "f" #'denote-query-filenames-link)
    (:prefix ("d" . "Dired")
      :desc "open dired"       "d" #'denote-dired
      :desc "link marked"      "i" #'denote-dired-link-marked-notes
      :desc "rename marked"    "r" #'denote-dired-rename-files
      :desc "rename + keyword" "k" #'denote-dired-rename-marked-files-with-keywords
      :desc "rename marked FM" "R" #'denote-dired-rename-marked-files-using-front-matter
    )
    (:prefix ("s" . "Sequence")
      :desc "new sequence"     "s" #'denote-sequence
      :desc "find sequence"    "f" #'denote-sequence-find
      :desc "link sequence"    "l" #'denote-sequence-link
      :desc "dired sequence"   "d" #'denote-sequence-dired)
    (:prefix ("j" . "Journal")
      :desc "new entry"        "j" #'denote-journal-new-entry
      :desc "new or existing"  "o" #'denote-journal-new-or-existing-entry
      :desc "link or create"   "l" #'denote-journal-link-or-create-entry))))
```

8 HTML Export

I want to tweak a whole bunch of things. While I'll want my tweaks almost all the time, occasionally I may want to test how something turns out using a more default config. With that in mind, a global minor mode seems like the most appropriate architecture to use.

```
(after! ox-html
```

```

(define-minor-mode org-fancy-html-export-mode
  "Toggle my fabulous org export tweaks. While this mode itself does a little bit,
the vast majority of the change in behaviour comes from switch statements in:
- 'org-html-template-fancier'
- 'org-html--build-meta-info-extended'
- 'org-html-src-block-collapsible'
- 'org-html-block-collapsible'
- 'org-html-table-wrapped'
- 'org-html--format-toc-headline-collapseable'
- 'org-html--toc-text-stripped-leaves'
- 'org-export-html-headline-anchor'"
  :global t
  :init-value t
  (if org-fancy-html-export-mode
      (setq org-html-style-default org-html-style-fancy
            org-html-meta-tags #'org-html-meta-tags-fancy
            org-html-checkbox-type 'html-span)
      (setq org-html-style-default org-html-style-plain
            org-html-meta-tags #'org-html-meta-tags-default
            org-html-checkbox-type 'html)))
)

```

8.1 Extra header contents

```

(after! ox-html
  (defadvice! org-html-template-fancier (orig-fn contents info)
    "Return complete document string after HTML conversion.
CONTENTS is the transcoded contents string. INFO is a plist
holding export options. Adds a few extra things to the body
compared to the default implementation."
    :around #'org-html-template
    (if (or (not org-fancy-html-export-mode) (bound-and-true-p org-msg-export-in-progress))
        (funcall orig-fn contents info)
        (concat
         (when (and (not (org-html-html5-p info)) (org-html-xhtml-p info))
           (let* ((xml-declaration (plist-get info :html-xml-declaration))
                  (decl (or (and (stringp xml-declaration) xml-declaration)
                             (cdr (assoc (plist-get info :html-extension)
                                           xml-declaration)))
                  (cdr (assoc "html" xml-declaration)))

```

```

        "")))
    (when (not (or (not decl) (string= "" decl)))
      (format "%s\n"
        (format decl
          (or (and org-html-coding-system
                    (fboundp 'coding-system-get)
                    (coding-system-get org-html-coding-system 'mime-
                      "iso-8859-1"))))))
    (org-html-doctype info)
    "\n"
    (concat "<html"
      (cond ((org-html-xhtml-p info)
        (format
          " xmlns=\"http://www.w3.org/1999/xhtml\" lang=\"%s\" xml:lang=\"
          (plist-get info :language) (plist-get info :language)))
        ((org-html-html5-p info)
          (format " lang=\"%s\" (plist-get info :language))))
      ">\n")
    "<head>\n"
    (org-html--build-meta-info info)
    (org-html--build-head info)
    (org-html--build-mathjax-config info)
    "</head>\n"
    "<body>\n<input type='checkbox' id='theme-switch'><div id='page'><label id='swi
    (let ((link-up (org-trim (plist-get info :html-link-up)))
      (link-home (org-trim (plist-get info :html-link-home))))
      (unless (and (string= link-up "") (string= link-home ""))
        (format (plist-get info :html-home/up-format)
          (or link-up link-home)
          (or link-home link-up))))
    ;; Preamble.
    (org-html--build-pre/postamble 'preamble info)
    ;; Document contents.
    (let ((div (assq 'content (plist-get info :html-divs))))
      (format "<%s id=\"%s\">\n" (nth 1 div) (nth 2 div)))
    ;; Document title.
    (when (plist-get info :with-title)
      (let ((title (and (plist-get info :with-title)
        (plist-get info :title)))
        (subtitle (plist-get info :subtitle)))

```

```

        (html5-fancy (org-html--html5-fancy-p info)))
    (when title
      (format
        (if html5-fancy
          "<header class=\"page-header\">%s\n<h1 class=\"title\">%s</h1>\n%s</h1>\n"
          "<h1 class=\"title\">%s</h1>\n")
        (if (or (plist-get info :with-date)
            (plist-get info :with-author))
          (concat "<div class=\"page-meta\">"
            (when (plist-get info :with-date)
              (org-export-data (plist-get info :date) info))
            (when (and (plist-get info :with-date) (plist-get info :with-author))
              (when (plist-get info :with-author)
                (org-export-data (plist-get info :author) info)))
            "</div>\n")
          "")
        (org-export-data title info)
        (if subtitle
          (format
            (if html5-fancy
              "<p class=\"subtitle\" role=\"doc-subtitle\">%s</p>\n"
              (concat "\n" (org-html-close-tag "br" nil info) "\n"
                "<span class=\"subtitle\">%s</span>\n"))
            (org-export-data subtitle info))
          ""))))))
  contents
  (format "</%s>\n" (nth 1 (assq 'content (plist-get info :html-divs))))
  ;; Postamble.
  (org-html--build-pre/postamble 'postamble info)
  ;; Possibly use the Klipse library live code blocks.
  (when (plist-get info :html-klipsify-src)
    (concat "<script>" (plist-get info :html-klipse-selection-script)
      "</script><script src=\""
      org-html-klipse-js
      "\"></script><link rel=\"stylesheet\" type=\"text/css\" href=\""
      org-html-klipse-css "\"/>"))
  ;; Closing document.
  "</div>\n</body>\n</html>"))))
)

```

```

(after! ox-html
  (defadvice! org-html-toc-linked (depth info &optional scope)
    "Build a table of contents.

    Just like 'org-html-toc', except the header is a link to \"#\".

    DEPTH is an integer specifying the depth of the table. INFO is
    a plist used as a communication channel. Optional argument SCOPE
    is an element defining the scope of the table. Return the table
    of contents as a string, or nil if it is empty."
    :override #'org-html-toc
    (let ((toc-entries
          (mapcar (lambda (headline)
                    (cons (org-html--format-toc-headline headline info)
                          (org-export-get-relative-level headline info)))
                  (org-export-collect-headlines info depth scope))))
      (when toc-entries
        (let ((toc (concat "<div id=\"text-table-of-contents\">"
                          (org-html--toc-text toc-entries)
                          "</div>\n")))
          (if scope toc
              (let ((outer-tag (if (org-html--html5-fancy-p info)
                                    "nav"
                                    "div")))
                (concat (format "<%s id=\"table-of-contents\">\n" outer-tag)
                        (let ((top-level (plist-get info :html-toplevel-hlevel)))
                          (format "<h%d><a href=\"#\" style=\"color:inherit; text-decoration: none;\">"
                                  top-level
                                  (org-html--translate "Table of Contents" info)
                                  top-level))
                          toc
                          (format "</%s>\n" outer-tag)))))))
      )

    (after! ox-html
      (defvar org-html-meta-tags-opengraph-image
        '(:image "https://tecosaur.com/resources/org/nib.png"
          :type "image/png"
          :width "200"
          :height "200")

```

```

:alt "Green fountain pen nib")
"Plist of og:image:PROP properties and their value, for use in 'org-html-meta-tags

(defun org-html-meta-tags-fancy (info)
  "Use the INFO plist to construct the meta tags, as described in 'org-html-meta-tags
  (let* ((title (org-html-plain-text
                 (org-element-interpret-data (plist-get info :title)) info))
         (author (and (plist-get info :with-author)
                      (let ((auth (plist-get info :author)))
                        ;; Return raw Org syntax.
                        (and auth (org-html-plain-text
                              (org-element-interpret-data auth) info))))))
         (author-first-last
          (and (not (org-string-nw-p author))
               (save-match-data
                 (if (string-match "\\['\\(.+?\\) +\\(.+?\\)\\'" author)
                     (cons (match-string 1 author)
                           (match-string 2 author))
                     (cons author nil))))))
         (append
          (list
           (when (org-string-nw-p author)
             (list "name" "author" author))
           (when (org-string-nw-p (plist-get info :description))
             (list "name" "description"
                   (plist-get info :description)))
           '("name" "generator" "org mode")
           '("name" "theme-color" "#77aa99")
           '("property" "og:type" "article")
           (list "property" "og:title" title)
           (let ((subtitle (org-export-data (plist-get info :subtitle) info)))
             (when (org-string-nw-p subtitle)
               (list "property" "og:description" subtitle))))
           (when org-html-meta-tags-opengraph-image
             (list (list "property" "og:image" (plist-get org-html-meta-tags-opengraph-image
                                                           (plist-get info :image)))
                   (list "property" "og:image:type" (plist-get org-html-meta-tags-opengraph-image
                                                                 (plist-get info :image-type)))
                   (list "property" "og:image:width" (plist-get org-html-meta-tags-opengraph-image
                                                                 (plist-get info :image-width)))
                   (list "property" "og:image:height" (plist-get org-html-meta-tags-opengraph-image
                                                                 (plist-get info :image-height)))
                   (list "property" "og:image:alt" (plist-get org-html-meta-tags-opengraph-image
                                                                 (plist-get info :image-alt))))))
          (list

```

```

    (when (car author-first-last)
      (list "property" "og:article:author:first_name" (car author-first-last)))
    (when (cdr author-first-last)
      (list "property" "og:article:author:last_name" (cdr author-first-last)))
    (list "property" "og:article:published_time"
      (format-time-string
        "%FT%T%Z"
        (or
          (when-let ((date-str (cadar (org-collect-keywords '("DATE")))))
            (unless (string= date-str (format-time-string "%F"))
              (ignore-errors (encode-time (org-parse-time-string date-str))))))
          (if buffer-file-name
              (file-attribute-modification-time (file-attributes buffer-file-name)
                (current-time))))))
    (when buffer-file-name
      (list "property" "og:article:modified_time"
        (format-time-string "%FT%T%Z" (file-attribute-modification-time (file-

```

```

(unless (functionp #'org-html-meta-tags-default)
  (defalias 'org-html-meta-tags-default #'ignore))
(setq org-html-meta-tags #'org-html-meta-tags-fancy)
)

```

8.2 Custom CSS/JS

The default org HTML export is ... alright, but we can really jazz it up. lepisma.xyz has a really nice style, and from an org export too! Suffice to say I've snatched it, with a few of my own tweaks applied.

```

<link rel="icon" href="https://tecosaur.com/resources/org/nib.ico" type="image/ico" />

<link rel="preload" as="font" crossorigin="anonymous" type="font/woff2" href="https://t
<link rel="preload" as="font" crossorigin="anonymous" type="font/woff2" href="https://t
<link rel="preload" as="font" crossorigin="anonymous" type="font/woff2" href="https://t
<link rel="preload" as="font" crossorigin="anonymous" type="font/woff2" href="https://t
<link rel="preload" as="font" crossorigin="anonymous" type="font/woff2" href="https://t

(after! ox-html
  (setq org-html-style-plain org-html-style-default
    org-html-htmlize-output-type 'css

```

```

org-html-doctype "html5"
org-html-html5-fancy t)

(defun org-html-reload-fancy-style ()
  "Rebuild 'org-html-style-fancy' from the asset files under misc/org-css/.
  If any asset is missing, keep the default style rather than erroring -
  run =doom sync= is not needed, just place the files and call this again."
  (interactive)
  (let ((header (expand-file-name "misc/org-export-header.html" doom-user-dir))
        (js      (expand-file-name "misc/org-css/main.js" doom-user-dir))
        (css      (expand-file-name "misc/org-css/main.min.css" doom-user-dir)))
    (if (not (and (file-exists-p header) (file-exists-p js) (file-exists-p css)))
        (when (called-interactively-p 'any)
            (message "org-html fancy style assets missing under %smisc/org-css/"
                     doom-user-dir))
        (setq org-html-style-fancy
              (with-temp-buffer
                (insert-file-contents header)
                (goto-char (point-max))
                (insert "<script>\n")
                (insert-file-contents js)
                (goto-char (point-max))
                (insert "</script>\n<style>\n")
                (insert-file-contents css)
                (goto-char (point-max))
                (insert "</style>")
                (buffer-string)))
        (when org-fancy-html-export-mode
          (setq org-html-style-default org-html-style-fancy))))
  (org-html-reload-fancy-style))

```

8.3 Collapsible src and example blocks

By wrapping the `<pre>` element in a `<details>` block, we can obtain collapsable blocks with no CSS, though we will toss a little in anyway to have this looking somewhat spiffy.

Since this collapsability seems useful to have on by default for certain chunks of code, it would be nice if you could set it with `#+attr_html: :collapsed t`.

It would be nice to also have a corresponding global / session-local way

of setting this, but I haven't quite been able to get that working (yet).

```
(after! ox-html
  (defvar org-html-export-collapsed nil)
  (eval '(cl-pushnew '(:collapsed "COLLAPSED" "collapsed" org-html-export-collapsed t)
    (org-export-backend-options (org-export-get-backend 'html))))
  (add-to-list 'org-default-properties "EXPORT_COLLAPSED")
)
```

We can take our src block modification a step further, and add a gutter on the side of the src block containing both an anchor referencing the current block, and a button to copy the content of the block.

```
(after! ox-html
  (defadvice! org-html-src-block-collapsible (orig-fn src-block contents info)
    "Wrap the usual <pre> block in a <details>"
    :around #'org-html-src-block
    (if (or (not org-fancy-html-export-mode) (bound-and-true-p org-msg-export-in-progress))
        (funcall orig-fn src-block contents info)
        (let* ((properties (cadr src-block))
               (lang (mode-name-to-lang-name
                      (plist-get properties :language)))
               (name (plist-get properties :name))
               (ref (org-export-get-reference src-block info))
               (collapsed-p (member (or (org-export-read-attribute :attr_html src-block
                                                                    (plist-get info :collapsed))
                                      '("y" "yes" "t" t "true" "all"))))
               (format
                "<details id='%s' class='code'%s><summary%s>%s</summary>"
                "<div class='gutter'>"
                "<a href='#%s'>#</a>"
                "<button title='Copy to clipboard' onclick='copyPreToClipboard(this)'></button>\n"
                "</div>"
                "%s"
                "</details>"
                ref
                (if collapsed-p "" " open")
                (if name " class='named'" ""))
               (concat
                (when name (concat "<span class=\"name\">" name "</span>"))
```

```

" <span class=\"lang\">" lang "</span>"
ref
(if name
  (replace-regexp-in-string (format "<pre\\( class=\"[^\"]+\"\\)? id=\"%s\"")
    (funcall orig-fn src-block contents info))
  (funcall orig-fn src-block contents info))))))

(defun mode-name-to-lang-name (mode)
  (or (cadr (assoc mode
    '(("asymptote" "Asymptote")
      ("awk" "Awk")
      ("C" "C")
      ("clojure" "Clojure")
      ("css" "CSS")
      ("D" "D")
      ("ditaa" "ditaa")
      ("dot" "Graphviz")
      ("calc" "Emacs Calc")
      ("emacs-lisp" "Emacs Lisp")
      ("fortran" "Fortran")
      ("gnuplot" "gnuplot")
      ("haskell" "Haskell")
      ("hledger" "hledger")
      ("java" "Java")
      ("js" "Javascript")
      ("latex" "LaTeX")
      ("ledger" "Ledger")
      ("lisp" "Lisp")
      ("lilypond" "Lilypond")
      ("lua" "Lua")
      ("matlab" "MATLAB")
      ("mscgen" "Mscgen")
      ("ocaml" "Objective Caml")
      ("octave" "Octave")
      ("org" "Org mode")
      ("oz" "OZ")
      ("plantuml" "Plantuml")
      ("processing" "Processing.js")
      ("python" "Python")
      ("R" "R"))
    )
    ))

```

```
("ruby" "Ruby")
("sass" "Sass")
("scheme" "Scheme")
("screen" "Gnu Screen")
("sed" "Sed")
("sh" "shell")
("sql" "SQL")
("sqlite" "SQLite")
("forth" "Forth")
("io" "IO")
("J" "J")
("makefile" "Makefile")
("maxima" "Maxima")
("perl" "Perl")
("picolisp" "Pico Lisp")
("scala" "Scala")
("shell" "Shell Script")
("ebnf2ps" "ebfn2ps")
("cpp" "C++")
("abc" "ABC")
("coq" "Coq")
("groovy" "Groovy")
("bash" "bash")
("csh" "csh")
("ash" "ash")
("dash" "dash")
("ksh" "ksh")
("mksh" "mksh")
("posh" "posh")
("ada" "Ada")
("asm" "Assembler")
("caml" "Caml")
("delphi" "Delphi")
("html" "HTML")
("idl" "IDL")
("mercury" "Mercury")
("metapost" "MetaPost")
("modula-2" "Modula-2")
("pascal" "Pascal")
("ps" "PostScript")
```

```

        ("prolog" "Prolog")
        ("simula" "Simula")
        ("tcl" "tcl")
        ("tex" "LaTeX")
        ("plain-tex" "TeX")
        ("verilog" "Verilog")
        ("vhdl" "VHDL")
        ("xml" "XML")
        ("nxml" "XML")
        ("conf" "Configuration File")
    )))
    mode))
)

(after! ox-html
  (defun org-html-block-collapsible (orig-fn block contents info)
    "Wrap the usual block in a <details>"
    (if (or (not org-fancy-html-export-mode) (bound-and-true-p org-msg-export-in-progress))
        (funcall orig-fn block contents info)
        (let ((ref (org-export-get-reference block info))
              (type (pcase (car block)
                        ('property-drawer "Properties"))
              (collapsed-default (pcase (car block)
                                         ('property-drawer t)
                                         (_ nil)))
              (collapsed-value (org-export-read-attribute :attr_html block :collapsed))
              (collapsed-p (or (member (org-export-read-attribute :attr_html block :collapsed)
                                       '("y" "yes" "t" t "true"))
                              (member (plist-get info :collapsed) '("all")))))
            (format
             "<details id='%s' class='code'%s>
<summary%s>%s</summary>
<div class='gutter'>\
<a href='#%s'>#</a>
<button title='Copy to clipboard' onclick='copyPreToClipboard(this)'></button>\
</div>
%s\n
</details>"
             ref
             (if (or collapsed-p collapsed-default) "" " open")
             (org-export-read-attribute :attr_html block :summary)
             (org-export-read-attribute :attr_html block :content)
             (org-export-read-attribute :attr_html block :pre)
             (org-export-read-attribute :attr_html block :post))
            (funcall orig-fn block contents info)))))

```

```

      (if type " class='named'" "")
      (if type (format "<span class='type'>%s</span>" type) "")
      ref
      (funcall orig-fn block contents info))))))

(advice-add 'org-html-example-block :around #'org-html-block-collapsible)
(advice-add 'org-html-fixed-width :around #'org-html-block-collapsible)
(advice-add 'org-html-property-drawer :around #'org-html-block-collapsible)
)

```

8.4 Handle table overflow

In order to accommodate wide tables —particularly on mobile devices— we want to set a maximum width and scroll overflow. Unfortunately, this cannot be applied directly to the `table` element, so we have to wrap it in a `div`.

While we're at it, we can add a link gutter, as we did with `src` blocks, and show the `#+name`, if one is given.

```

(after! ox-html
  (defadvice! org-html-table-wrapped (orig-fn table contents info)
    "Wrap the usual <table> in a <div>"
    :around #'org-html-table
    (if (or (not org-fancy-html-export-mode) (bound-and-true-p org-msg-export-in-progress))
      (funcall orig-fn table contents info)
      (let* ((name (plist-get (cadr table) :name))
             (ref (org-export-get-reference table info)))
        (format "<div id='%s' class='table'>
<div class='gutter'><a href='%s'>#</a></div>
<div class='tabular'>
%s
</div>\\
</div>"
              ref ref
              (if name
                (replace-regexp-in-string (format "<table id=\\\"%s\\\"" ref) "<table"
                                           (funcall orig-fn table contents info))
                (funcall orig-fn table contents info))))))
  )

```

8.5 TOC as a collapsable tree

The TOC is much nicer to navigate as a collapsable tree. Unfortunately we cannot achieve this with CSS alone. Thankfully we can avoid JS though, by adapting the TOC generation code to use a `label` for each item, and a hidden `checkbox` to keep track of state.

To add this, we need to change one line in `org-html--format-toc-headline`. Since we can actually accomplish the desired effect by adding advice *around* the function, without overriding it — let's do that to reduce the bug surface of this config a tad.

```
(after! ox-html
  (defadvice! org-html--format-toc-headline-collapsible (orig-fn headline info)
    "Add a label and checkbox to 'org-html--format-toc-headline's usual output,
    to allow the TOC to be a collapsable tree."
    :around #'org-html--format-toc-headline
    (if (or (not org-fancy-html-export-mode) (bound-and-true-p org-msg-export-in-progress))
        (funcall orig-fn headline info)
        (let ((id (or (org-element-property :CUSTOM_ID headline)
                     (org-export-get-reference headline info))))
          (format "<input type='checkbox' id='toc--%s' /><label for='toc--%s'>%s</label>"
                  id id (funcall orig-fn headline info))))))
```

Now, leaves (headings with no children) shouldn't have the `label` item. The obvious way to achieve this is by including some *if no children...* logic in `org-html--format-toc-headline-collapsible`. Unfortunately, I can't my elisp isn't up to par to extract the number of child headings from the mountain of info that org provides.

```
(after! ox-html
  (defadvice! org-html--toc-text-stripped-leaves (orig-fn &rest args)
    "Remove label"
    :around #'org-html--toc-text
    (if (or (not org-fancy-html-export-mode) (bound-and-true-p org-msg-export-in-progress))
        (apply orig-fn args)
        (replace-regexp-in-string "<input [^>]+><label [^>]+>\\(\\.\\+?\\)\\</label></li>" "\\</li>"
                                   (apply orig-fn args)))))
```

8.6 Make verbatim different to code

Since we have `verbatim` and `code`, let's make use of the difference. We can use `code` exclusively for code snippets and commands like: `Hello` in batch-

mode Emacs prints to stdout. Then we can use `verbatim` for miscellaneous 'other monospace' like keyboard shortcuts: "either `C-c C-c` or `C-g` is likely the most useful keybinding in Emacs", or file names: "I keep my configuration in `~/.config/doom/`", among other things.

Then, styling these two cases differently can help improve clarity in a document.

```
(after! ox-html
  (setq org-html-text-markup-alist
    '( (bold . "<b>%s</b>")
      (code . "<code>%s</code>")
      (italic . "<i>%s</i>")
      (strike-through . "<del>%s</del>")
      (underline . "<span class='underline'>%s</span>")
      (verbatim . "<kbd>%s</kbd>"))))
```

8.7 Change checkbox type

We also want to use HTML checkboxes, however we want to get a bit fancier than default

```
(after! ox-html
  (appendq! org-html-checkbox-types
    '( (html-span
      (on . "<span class='checkbox'></span>")
      (off . "<span class='checkbox'></span>")
      (trans . "<span class='checkbox'></span>"))))
  (setq org-html-checkbox-type 'html-span))
```

8.8 Extra special strings

The `org-html-special-string-regexps` variable defines substitutions for:

- `\-`, a shy hyphen
- `---`, an em dash
- `--`, an en dash
- `...`, (horizontal) ellipses

However I think it would be nice if there was also a substitution for left/right arrows ($\rightarrow$ and $\leftarrow$). This is a `defconst`, but as you may tell from the amount of advice in this config, I'm not above messing with things I'm not 'supposed' to.

The only minor complication is that $<$ and $>$ are converted to `<` and `>` before this stage of output processing.

```
(after! ox-html
  (pushnew! org-html-special-string-regexps
    '("-&gt;" . "&#8594;")
    '("&lt;-" . "&#8592;"))
)
```

8.9 Header anchors

I want to add GitHub-style links on hover for headings.

```
(after! ox-html
  (defun org-export-html-headline-anchor (text backend info)
    (when (and (org-export-derived-backend-p backend 'html)
               (not (org-export-derived-backend-p backend 're-reveal))
               org-fancy-html-export-mode)
      (unless (bound-and-true-p org-msg-export-in-progress)
        (replace-regexp-in-string
          "<h\\([0-9]\\) id=\\\"\\([a-z0-9-]+\\)\\\">\\\"(.\\*[~ ]\\)\\\"<\\\"/h[0-9]>\" ; this is quiet
          "<h\\1 id=\\\"\\2\\\">\\3<a aria-hidden=\\\"true\\\" href=\\\"#\\2\\\">#</a> </h\\1>\"
          text))))

  (add-to-list 'org-export-filter-headline-functions
    'org-export-html-headline-anchor)
)
```

8.10 L^AT_EX Rendering

8.10.1 Pre-rendered

I consider `dvisvgm` to be a rather compelling option. However this isn't scaled very well at the moment.

```
(after! ox-html
  ;; (setq-default org-html-with-latex 'dvisvgm)
)
```


8.10.2 MathJax

I want to use svg MathJax by default, and with a few of the custom commands that are part of my $\text{\LaTeX}$ preamble.

```
(after! ox-html
  (setcdr (assoc 'path org-html-mathjax-options)
    (list "https://cdn.jsdelivr.net/npm/mathjax@3/es5/tex-svg.js")))

(setq org-html-mathjax-template
  "<script>
    <script>
window.MathJax = {
  loader: {
    load: ['[tex]/mathtools'],
  },
  tex: {
    ams: {
      multlineWidth: '%MULTLINEWIDTH'
    },
    tags: '%TAGS',
    tagSide: '%TAGSIDE',
    tagIndent: '%TAGINDENT',
    packages: {'[+]': ['mathtools']},
    macros: {
      RR: ['\\\\\\ifstrempy{#1}{\\\\\\mathbb{R}}{\\\\\\mathbb{R}^{\{#1\}}', 1, ''],
      NN: ['\\\\\\ifstrempy{#1}{\\\\\\mathbb{N}}{\\\\\\mathbb{N}^{\{#1\}}', 1, ''],
      ZZ: ['\\\\\\ifstrempy{#1}{\\\\\\mathbb{Z}}{\\\\\\mathbb{Z}^{\{#1\}}', 1, ''],
      QQ: ['\\\\\\ifstrempy{#1}{\\\\\\mathbb{Q}}{\\\\\\mathbb{Q}^{\{#1\}}', 1, ''],
      CC: ['\\\\\\ifstrempy{#1}{\\\\\\mathbb{C}}{\\\\\\mathbb{C}^{\{#1\}}', 1, ''],
      EE: '\\\\\\mathbb{E}',
      Lap: '\\\\\\operatorname{\\\\\\mathcal{L}}',
      Var: '\\\\\\operatorname{Var}',
      Cor: '\\\\\\operatorname{Cor}',
      E: '\\\\\\operatorname{E}',
    },
  },
  mathtools: {
    pairedDelimiters: {
      abs: ['\\\\\\lvert', '\\\\\\rvert'],
      norm: ['\\\\\\lVert', '\\\\\\rVert'],
      ceil: ['\\\\\\lceil', '\\\\\\rceil'],
```

```

        floor: ['\\\\\\lfloor', '\\\\\\rfloor'],
        round: ['\\\\\\lfloor', '\\\\\\rceil'],
    }
}
},
chtml: {
    scale: %SCALE,
    displayAlign: '%ALIGN',
    displayIndent: '%INDENT'
},
svg: {
    scale: %SCALE,
    displayAlign: '%ALIGN',
    displayIndent: '%INDENT'
},
output: {
    font: '%FONT',
    displayOverflow: '%OVERFLOW'
}
};
</script>

<script
    id="MathJax-script\"
    async
    src=\"%PATH\">
</script>")
)

```